REFERENCE

Plugins reference

Complete technical reference for Claude Code plugin system, including schemas, CLI commands, and component specifications.

Looking to install plugins? See **Discover and install plugins**. For creating plugins, see **Plugins**. For distributing plugins, see **Plugin marketplaces**.

This reference provides complete technical specifications for the Claude Code plugin system, including component schemas, CLI commands, and development tools.

A **plugin** is a self-contained directory of components that extends Claude Code with custom functionality. Plugin components include skills, agents, hooks, MCP servers, LSP servers, and monitors.

Plugin components reference

Skills

Plugins add skills to Claude Code, creating `/name` shortcuts that you or Claude can invoke.

Location: `skills/` or `commands/` directory in plugin root, or a single `SKILL.md` file at the plugin root

File format: Skills are directories with `SKILL.md`; commands are simple markdown files

Skill structure:

```
skills/
├── pdf-processor/
│   ├── SKILL.md
│   ├── reference.md (optional)
│   └── scripts/ (optional)
└── code-reviewer/
    └── SKILL.md
```

Integration behavior:

- Skills and commands are automatically discovered when the plugin is installed
- Claude can invoke them automatically based on task context
- Skills can include supporting files alongside SKILL.md

If a plugin has no `skills/` directory and no `skills` manifest field, a `SKILL.md` at the plugin root is loaded as a single skill. Set the frontmatter `name` field to control the skill's invocation name. Without it, Claude Code falls back to the install directory name, which for marketplace-installed plugins is a version string that changes on every update. For plugins that ship more than one skill, use the `skills/` directory layout shown above.

For complete details, see **Skills**.

Agents

Plugins can provide specialized subagents for specific tasks that Claude can invoke automatically when appropriate.

Location: `agents/` directory in plugin root

File format: Markdown files describing agent capabilities

Agent structure:

```
---
name: agent-name
description: What this agent specializes in and when Claude
should invoke it
model: sonnet
effort: medium
maxTurns: 20
disallowedTools: Write, Edit
---

Detailed system prompt for the agent describing its role,
expertise, and behavior.
```

Plugin agents support `name`, `description`, `model`, `effort`, `maxTurns`, `tools`, `disallowedTools`, `skills`, `memory`, `background`, and `isolation` frontmatter fields. The only valid `isolation` value is `"worktree"`. For security reasons, `hooks`, `mcpServers`, and `permissionMode` are not supported for plugin-shipped agents.

Integration points:

- Agents appear in the `/agents` interface
- Claude can invoke agents automatically based on task context
- Agents can be invoked manually by users
- Plugin agents work alongside built-in Claude agents

For complete details, see **Subagents**.

Hooks

Plugins can provide event handlers that respond to Claude Code events automatically.

Location: `hooks/hooks.json` in plugin root, or inline in plugin.json

Format: JSON configuration with event matchers and actions

Hook configuration:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command": "\"${CLAUDE_PLUGIN_ROOT}\"/scripts/format-
code.sh"
          }
        ]
      }
    ]
  }
}
```

Plugin hooks respond to the same lifecycle events as **user-defined hooks**:

Event	When it fires
`SessionStart`	When a session begins or resumes
`Setup`	When you start Claude Code with `--init-only`, or with `--init` or `--maintenance` in `-p` mode. For one-time preparation in CI or scripts
`UserPromptSubmit`	When you submit a prompt, before Claude processes it
`UserPromptExpansion`	When a user-typed command expands into a prompt, before it reaches Claude. Can block the expansion
`PreToolUse`	Before a tool call executes. Can block it
`PermissionRequest`	When a permission dialog appears
`PermissionDenied`	When a tool call is denied by the auto mode classifier. Return `{retry: true}` to tell the model it may retry the denied tool call
`PostToolUse`	After a tool call succeeds
`PostToolUseFailure`	After a tool call fails
`PostToolBatch`	After a full batch of parallel tool calls resolves, before the next model call
`Notification`	When Claude Code sends a notification
`MessageDisplay`	While assistant message text is displayed
`SubagentStart`	When a subagent is spawned

Event	When it fires
`SubagentStop`	When a subagent finishes
`TaskCreated`	When a task is being created via `TaskCreate`
`TaskCompleted`	When a task is being marked as completed
`Stop`	When Claude finishes responding
`StopFailure`	When the turn ends due to an API error. Output and exit code are ignored
`TeammateIdle`	When an **agent team** teammate is about to go idle
`InstructionsLoaded`	When a CLAUDE.md or `.claude/rules/*.md` file is loaded into context. Fires at session start and when files are lazily loaded during a session
`ConfigChange`	When a configuration file changes during a session
`CwdChanged`	When the working directory changes, for example when Claude executes a `cd` command. Useful for reactive environment management with tools like direnv
`FileChanged`	When a watched file changes on disk. The `matcher` field specifies which filenames to watch
`WorktreeCreate`	When a worktree is being created via `--worktree` or `isolation: "worktree"`. Replaces default git behavior
`WorktreeRemove`	When a worktree is being removed, either at session exit or when a subagent finishes
`PreCompact`	Before context compaction
`PostCompact`	After context compaction completes
`Elicitation`	When an MCP server requests user input during a tool call
`ElicitationResult`	After a user responds to an MCP elicitation, before the response is sent back to the server
`SessionEnd`	When a session terminates

Hook types:

- `command` : execute shell commands or scripts
- `http` : send the event JSON as a POST request to a URL
- `mcp_tool` : call a tool on a configured **MCP server**
- `prompt` : evaluate a prompt with an LLM (uses `$ARGUMENTS` placeholder for context)
- `agent` : run an agentic verifier with tools for complex verification tasks

MCP servers

Plugins can bundle Model Context Protocol (MCP) servers to connect Claude Code with external tools and services.

Location: `.mcp.json` in plugin root, or inline in plugin.json

Format: Standard MCP server configuration

MCP server configuration:

```
{
  "mcpServers": {
    "plugin-database": {
      "command": "${CLAUDE_PLUGIN_ROOT}/servers/db-server",
      "args": ["--config", "${CLAUDE_PLUGIN_ROOT}/config.json"],
      "env": {
        "DB_PATH": "${CLAUDE_PLUGIN_ROOT}/data"
      }
    },
    "plugin-api-client": {
      "command": "npx",
      "args": ["@company/mcp-server", "--plugin-mode"],
      "cwd": "${CLAUDE_PLUGIN_ROOT}"
    }
  }
}
```

Integration behavior:

- Plugin MCP servers start automatically when the plugin is enabled
- Servers appear as standard MCP tools in Claude's toolkit
- Server capabilities integrate seamlessly with Claude's existing tools
- Plugin servers can be configured independently of user MCP servers

LSP servers

Looking to use LSP plugins? Install them from the official marketplace: search for "lsp" in the `/plugin` Discover tab. This section documents how to create LSP plugins for languages not covered by the official marketplace.

Plugins can provide **Language Server Protocol** (LSP) servers to give Claude real-time code intelligence while working on your codebase.

LSP integration provides:

- **Instant diagnostics**: Claude sees errors and warnings immediately after each edit
- **Code navigation**: go to definition, find references, and hover information
- **Language awareness**: type information and documentation for code symbols

Location: `.lsp.json` in plugin root, or inline in `plugin.json`

Format: JSON configuration mapping language server names to their configurations

`.lsp.json` file format:

```
{
  "go": {
    "command": "gopls",
    "args": ["serve"],
    "extensionToLanguage": {
      ".go": "go"
    }
  }
}
```

Inline in `plugin.json`:

```
{
  "name": "my-plugin",
  "lspServers": {
    "go": {
      "command": "gopls",
      "args": ["serve"],
      "extensionToLanguage": {
        ".go": "go"
      }
    }
  }
}
```

Required fields:

Field	Description
`command`	The LSP binary to execute (must be in PATH)
`extensionToLanguage`	Maps file extensions to language identifiers

Optional fields:

Field	Description
`args`	Command-line arguments for the LSP server
`transport`	Communication transport: `stdio` (default) or `socket`
`env`	Environment variables to set when starting the server
`initializationOptions`	Options passed to the server during initialization
`settings`	Settings passed via `workspace/didChangeConfiguration`
`workspaceFolder`	Workspace folder path for the server
`startupTimeout`	Max time to wait for server startup (milliseconds)
`maxRestarts`	Maximum number of restart attempts before giving up
`diagnostics`	Whether to push diagnostics into Claude's context after edits (default `true`). Set to `false` to keep code navigation but suppress automatic diagnostic injection.

⚠ **You must install the language server binary separately.** LSP plugins configure how Claude Code connects to a language server, but they don't include the server itself. If you see `Executable not found in $PATH` in the `/plugin` Errors tab, install the required binary for your language.

Available LSP plugins:

Plugin	Language server	Install command
`pyright-lsp`	Pyright (Python)	`pip install pyright` or `npm install -g pyright`
`typescript-lsp`	TypeScript Language Server	`npm install -g typescript-language-server typescript`
`rust-analyzer-lsp`	rust-analyzer	**See rust-analyzer installation**

Install the language server first, then install the plugin from the marketplace.

Monitors

Plugins can declare background monitors that Claude Code starts automatically when the plugin is active. Each monitor runs a shell command for the lifetime of the session and delivers every stdout line to Claude as a notification, so Claude can react to log entries, status changes, or polled events without being asked to start the watch itself.

Plugin monitors use the same mechanism as the **Monitor tool** and share its availability constraints. They run only in interactive CLI sessions, run unsandboxed at the same trust level as **hooks**, and are skipped on hosts where the Monitor tool is unavailable.

> Plugin monitors require Claude Code v2.1.105 or later.

Location: `monitors/monitors.json` in the plugin root, or inline in `plugin.json`

Format: JSON array of monitor entries

The following `monitors/monitors.json` watches a deployment status endpoint and a local error log:

```
[
  {
    "name": "deploy-status",
    "command": "\"${CLAUDE_PLUGIN_ROOT}\"/scripts/poll-deploy.sh
${user_config.api_endpoint}",
    "description": "Deployment status changes"
  },
  {
    "name": "error-log",
    "command": "tail -F ./logs/error.log",
    "description": "Application error log",
    "when": "on-skill-invoke:debug"
  }
]
```

To declare monitors inline, set `experimental.monitors` in `plugin.json` to the same array. To load from a non-default path, set `experimental.monitors` to a relative path string such as `"./config/monitors.json"`. Monitors are an **experimental component**.

Required fields:

Field	Description
`name`	Identifier unique within the plugin. Prevents duplicate processes when the plugin reloads or a skill is invoked again
`command`	Shell command run as a persistent background process in the session working directory
`description`	Short summary of what is being watched. Shown in the task panel and in notification summaries

Optional fields:

Field	Description
`when`	Controls when the monitor starts. `"always"` starts it at session start and on plugin reload, and is the default. `"on-skill-invoke:<skill-name>"` starts it the first time the named skill in this plugin is dispatched

The `command` value supports the same **variable substitutions** as MCP and LSP server configs: `${CLAUDE_PLUGIN_ROOT}`, `${CLAUDE_PLUGIN_DATA}`, `${CLAUDE_PROJECT_DIR}`, `${user_config.*}`, and any `${ENV_VAR}` from the environment. Prefix the command with `cd "${CLAUDE_PLUGIN_ROOT}" &&` if the script needs to run from the plugin's own directory.

Disabling a plugin mid-session does not stop monitors that are already running. They stop when the session ends.

Themes

Plugins can ship color themes that appear in `/theme` alongside the built-in presets and the user's local themes. A theme is a JSON file in `themes/` with a `base` preset and a sparse `overrides` map of color tokens. Themes are an **experimental component**.

```
{
  "name": "Dracula",
  "base": "dark",
  "overrides": {
    "claude": "#bd93f9",
    "error": "#ff5555",
    "success": "#50fa7b"
  }
}
```

Selecting a plugin theme persists `custom:<plugin-name>:<slug>` in the user's config. Plugin themes are read-only; pressing `Ctrl+E` on one in `/theme` copies it into `~/.claude/themes/` so the user can edit the copy.

Plugin installation scopes

When you install a plugin, you choose a **scope** that determines where the plugin is available and who else can use it:

Scope	Settings file	Use case
`user`	`~/.claude/settings.json`	Personal plugins available across all projects (default)
`project`	`.claude/settings.json`	Team plugins shared via version control
`local`	`.claude/settings.local.json`	Project-specific plugins, gitignored
`managed`	**Managed settings**	Managed plugins (read-only, update only)

Plugins use the same scope system as other Claude Code configurations. For installation instructions and scope flags, see **Install plugins**. For a complete explanation of scopes, see **Configuration scopes**.

Skills-directory plugins

Any folder under a skills directory that contains a `.claude-plugin/plugin.json` manifest is loaded as a plugin named `<name>@skills-dir` on the next session, with no marketplace and no install step. Scaffold one with `plugin init`. Unlike a marketplace install, the plugin is discovered in place rather than copied into the plugin cache.

A skills directory tree supports three distinct things:

What you have	What it is
`<skills-dir>/foo/SKILL.md` with no manifest	A plain **skill** named `foo`
`<skills-dir>/foo/.claude-plugin/plugin.json`	A plugin `foo@skills-dir`, which can bundle its own skills, agents, hooks, and more

What you have	What it is
`<plugin>/skills/bar/SKILL.md`	A skill `bar` packaged inside a plugin

Choose where the plugin loads from

Skills directory	Scope	Loads
`~/.claude/skills/`	personal	In every project, since the location is yours alone
`<cwd>/.claude/skills/`	project	Only after you accept the workspace **trust dialog** for that folder

A project-scope plugin is checked into the repository and reaches every collaborator who clones it. Because that content comes from the repository rather than from you, it loads only after the same trust gate that governs `.claude/settings.json`, and components that run code are restricted further:

- MCP servers it declares go through the **same per-server approval** as a project `.mcp.json`
- LSP servers start only after you trust the workspace
- **Background monitors** do not load

Personal-scope plugins have none of these restrictions.

> ⚠ Project-scope `@skills-dir` plugins load only from the `.claude/skills/` of the directory where you start Claude Code. They do not **walk up to the repository root** the way plain skills and commands do, so launching from a subdirectory misses a plugin that lives at the repo root. Launch from the repository root, or run `/reload-plugins` after changing directories.

Edit, reload, and disable a skills-directory plugin

Changes you make to a skill's `SKILL.md` take effect immediately in the current session. Changes to the plugin's other components, such as `hooks/`, `.mcp.json`, `agents/`, and `output-styles/`, do not. Run `/reload-plugins` or restart Claude Code to pick those up. See **Live change detection**.

To stop loading a skills-directory plugin, delete its folder or disable it by name. There is no `uninstall` step because nothing was installed from a marketplace.

```
claude plugin disable my-tool@skills-dir
```

Plugin manifest schema

The `.claude-plugin/plugin.json` file defines your plugin's metadata and configuration. This section documents all supported fields and options.

The manifest is optional. If omitted, Claude Code auto-discovers components in **default locations** and derives the plugin name from the directory name. Use a manifest when you need to provide metadata or custom component paths.

Complete schema

```
{
  "name": "plugin-name",
  "displayName": "Plugin Name",
  "version": "1.2.0",
  "description": "Brief plugin description",
  "author": {
    "name": "Author Name",
    "email": "author@example.com",
    "url": "https://github.com/author"
  },
  "homepage": "https://docs.example.com/plugin",
  "repository": "https://github.com/author/plugin",
  "license": "MIT",
  "keywords": ["keyword1", "keyword2"],
  "skills": "./custom/skills/",
  "commands": ["./custom/commands/special.md"],
  "agents": ["./custom/agents/reviewer.md"],
  "hooks": "./config/hooks.json",
  "mcpServers": "./mcp-config.json",
  "outputStyles": "./styles/",
  "lspServers": "./.lsp.json",
  "experimental": {
    "themes": "./themes/",
    "monitors": "./monitors.json"
  },
  "dependencies": [
    "helper-lib",
    { "name": "secrets-vault", "version": "~2.1.0" }
  ]
}
```

Required fields

If you include a manifest, `name` is the only required field.

Field	Type	Description	Example
`name`	string	Unique identifier (kebab-case, no spaces)	`"deployment-tools"`

This name is used for namespacing components. For example, in the UI, the agent `agent-creator`

for the plugin with name `plugin-dev` will appear as `plugin-dev:agent-creator`.

Unrecognized fields

Claude Code ignores top-level fields it does not recognize. You can keep metadata from another ecosystem in `plugin.json` and the plugin still loads. This makes it practical to maintain one manifest that doubles as a VS Code or Cursor extension manifest, an npm `package.json`, or an MCPB/DXT bundle manifest.

`claude plugin validate` reports unrecognized fields as warnings, not errors. If a field is one or two characters off from a recognized one, the warning suggests the likely intended name. A plugin with only unrecognized-field warnings still passes validation and loads at runtime.

Fields with the wrong type still fail. For example, a `keywords` value that is a string instead of an array is a load error, and `claude plugin validate` reports it as one.

Pass `--strict` to treat warnings as errors. Use it in CI to catch a misspelled field name or a field left over from another tool's manifest before publishing, even though the plugin would load at runtime.

```
claude plugin validate ./my-plugin --strict
```

Metadata fields

Field	Type	Description	Example
`$schema`	string	JSON Schema URL for editor autocomplete and validation. Claude Code ignores this field at load time.	`"https://json.schemastore.org/claude-code-plugin-manifest.json"`
`displayName`	string	Human-readable name shown in the `/plugin` picker and other UI surfaces. Falls back to `name` when omitted. Unlike `name`, may contain spaces and any casing. Not used for namespacing or lookup. Requires Claude Code v2.1.143 or later.	`"Deployment Tools"`
`version`	string	Optional. Semantic version. Setting this pins the plugin to that version string, so users only receive updates when you bump it. If omitted, Claude Code falls back to the git commit SHA, so every commit is treated as a new version. If	`"2.1.0"`

Field	Type	Description	Example
		also set in the marketplace entry, `plugin.json` wins. See **Version management**.	
`description`	string	Brief explanation of plugin purpose	`"Deployment automation tools"`
`author`	object	Author information	`{"name": "Dev Team", "email": "dev@company.com"}`
`homepage`	string	Documentation URL	`"https://docs.example.com"`
`repository`	string	Source code URL	`"https://github.com/user/plugin"`
`license`	string	License identifier	`"MIT"`, `"Apache-2.0"`
`keywords`	array	Discovery tags	`["deployment", "ci-cd"]`
`defaultEnabled`	boolean	Whether the plugin starts in an enabled state when the user has not set one. Defaults to `true`. See **Default enablement**. Requires Claude Code v2.1.154 or later.	`false`

Default enablement

Set `defaultEnabled: false` in `plugin.json` to ship a plugin that installs disabled. The user turns it on with `claude plugin enable <plugin>` or the `/plugin` interface. Use this for plugins that add cost or scope a user should opt into, such as one that connects to an external service. This requires Claude Code v2.1.154 or later. Earlier versions ignore the field and enable the plugin on install.

`defaultEnabled` is the fallback when nothing else has decided the plugin's state. Two things take precedence over it:

- **The user's setting**: an entry for the plugin in `enabledPlugins` at any settings scope. Once written, it persists across plugin updates and reinstalls, so changing `defaultEnabled` in a later release does not flip an existing user.
- **A dependency requirement**: when a plugin is required by another one that is active, Claude Code writes `true` for it at install or enable time. That gives it an explicit setting, so its own default no longer applies. See **Enable or disable a plugin with dependencies**.

The same field can appear in a plugin's marketplace entry, where it takes precedence over the value in `plugin.json`. See **Optional plugin fields**.

Component path fields

Field	Type	Description	Example
`skills`	string|array	Custom skill directories containing `<name>/SKILL.md`. Adds to the default `skills/` scan. See **Path behavior rules** for the marketplace-root exception	`"./custom/skills/"`
`commands`	string|array	Custom flat `.md` skill files or directories (replaces default `commands/`)	`"./custom/cmd.md"` or `["./cmd1.md"]`
`agents`	string|array	Custom agent files (replaces default `agents/`)	`"./custom/agents/reviewer.md"`
`hooks`	string|array|object	Hook config paths or inline config	`"./my-extra-hooks.json"`
`mcpServers`	string|array|object	MCP config paths or inline config	`"./my-extra-mcp-config.json"`
`outputStyles`	string|array	Custom output style files/directories (replaces default `output-styles/`)	`"./styles/"`
`lspServers`	string|array|object	**Language Server Protocol** configs for code intelligence (go to definition, find references, etc.)	`"./.lsp.json"`
`experimental.themes`	string|array	Color theme files/directories (replaces default `themes/`). See **Themes**	`"./themes/"`
`experimental.monitors`	string|array	Background **Monitor** configurations that start automatically when the plugin is active. See **Monitors**	`"./monitors.json"`
`userConfig`	object	User-configurable values prompted at enable time. See **User configuration**	See below
`channels`	array	Channel declarations for message injection (Telegram, Slack, Discord style). See **Channels**	See below
`dependencies`	array	Other plugins this plugin requires, optionally with semver version constraints. See **Constrain plugin dependency versions**	`[{ "name": "secrets-vault", "version": "~2.1.0" }]`

Experimental components

Components under the `experimental` key, `themes` and `monitors`, have a manifest schema that may change between releases while they stabilize. Where you declare them is a separate migration: the top level still works, `claude plugin validate` warns, and a future release will require `experimental.*`.

User configuration

The `userConfig` field declares values that Claude Code prompts the user for when the plugin is enabled. Use this instead of requiring users to hand-edit `settings.json`.

```
{
  "userConfig": {
    "api_endpoint": {
      "type": "string",
      "title": "API endpoint",
      "description": "Your team's API endpoint"
    },
    "api_token": {
      "type": "string",
      "title": "API token",
      "description": "API authentication token",
      "sensitive": true
    }
  }
}
```

Keys must be valid identifiers. Each option supports these fields:

Field	Required	Description
`type`	Yes	One of `string`, `number`, `boolean`, `directory`, or `file`
`title`	Yes	Label shown in the configuration dialog
`description`	Yes	Help text shown beneath the field
`sensitive`	No	If `true`, masks input and stores the value in secure storage instead of `settings.json`
`required`	No	If `true`, validation fails when the field is empty
`default`	No	Value used when the user provides nothing

Field	Required	Description
`multiple`	No	For `string` type, allow an array of strings
`min` / `max`	No	Bounds for `number` type

Each value is available for substitution as `${user_config.KEY}` in MCP and LSP server configs, hook commands, and monitor commands. Non-sensitive values can also be substituted in skill and agent content. All values are exported to plugin subprocesses as `CLAUDE_PLUGIN_OPTION_<KEY>` environment variables.

Non-sensitive values are stored in `settings.json` under `pluginConfigs[<plugin-id>].options`. Sensitive values go to the system keychain (or `~/.claude/.credentials.json` where the keychain is unavailable). Keychain storage is shared with OAuth tokens and has an approximately 2 KB total limit, so keep sensitive values small.

Channels

The `channels` field lets a plugin declare one or more message channels that inject content into the conversation. Each channel binds to an MCP server that the plugin provides.

```
{
  "channels": [
    {
      "server": "telegram",
      "userConfig": {
        "bot_token": {
          "type": "string",
          "title": "Bot token",
          "description": "Telegram bot token",
          "sensitive": true
        },
        "owner_id": {
          "type": "string",
          "title": "Owner ID",
          "description": "Your Telegram user ID"
        }
      }
    }
  ]
}
```

The `server` field is required and must match a key in the plugin's `mcpServers`. The optional per-channel `userConfig` uses the same schema as the top-level field, letting the plugin prompt for bot tokens or owner IDs when the plugin is enabled.

Path behavior rules

Whether a custom path replaces or extends the plugin's default directory depends on the field:

- **Replaces the default**: `commands`, `agents`, `outputStyles`, `experimental.themes`, `experimental.monitors`. For example, when the manifest specifies `commands`, the default `commands/` directory is not scanned. To keep the default and add more, list it explicitly: `"commands": ["./commands/", "./extras/"]`
- **Adds to the default**: `skills`. The default `skills/` directory is always scanned, and directories listed in `skills` are loaded alongside it. Exception: for a **marketplace entry whose `source` resolves to the marketplace root**, declaring specific subdirectories replaces the default `skills/` scan
- **Own merge rules**: **hooks**, **MCP servers**, and **LSP servers**. See each section for how multiple sources combine

When a plugin has both a default folder and the matching manifest key, Claude Code v2.1.140 and later flags the ignored folder in `/doctor`, `claude plugin list`, and the `/plugin` detail view. The plugin still loads using the manifest paths. No warning is shown when the manifest key points into the default folder, for example `"commands": ["./commands/deploy.md"]`, because the folder is addressed explicitly in that case.

For all path fields:

- All paths must be relative to the plugin root and start with `./`
- Components from custom paths use the same naming and namespacing rules
- Multiple paths can be specified as arrays
- When a skill path points to a directory that contains a `SKILL.md` directly, for example `"skills": ["./"]` pointing to the plugin root, the frontmatter `name` field in `SKILL.md` determines the skill's invocation name. This gives a stable name regardless of the install directory. If `name` is not set in the frontmatter, the directory basename is used as a fallback.

A plugin that has a `SKILL.md` at its root, no `skills/` subdirectory, and no `skills` manifest field is automatically loaded as a single-skill plugin in Claude Code v2.1.142 and later. You do not need to set `"skills": ["./"]` in `plugin.json` for this layout. The skill's invocation name follows the same

rule as above: the frontmatter `name` field, or the directory basename as a fallback.

Path examples:

```
{
  "commands": [
    "./specialized/deploy.md",
    "./utilities/batch-process.md"
  ],
  "agents": [
    "./custom-agents/reviewer.md",
    "./custom-agents/tester.md"
  ]
}
```

Environment variables

Claude Code provides three variables for referencing paths. All are substituted inline anywhere they appear in skill content, agent content, hook commands, monitor commands, and MCP or LSP server configs. All are also exported as environment variables to hook processes and MCP or LSP server subprocesses.

`${CLAUDE_PLUGIN_ROOT}` : the absolute path to your plugin's installation directory. Use this to reference scripts, binaries, and config files bundled with the plugin. In hook commands, use **exec form** with `args` so the path is passed as one argument with no quoting. In shell-form hooks and monitor commands, wrap it in double quotes, as in `"${CLAUDE_PLUGIN_ROOT}"`. This path changes when the plugin updates. The previous version's directory remains on disk for about seven days after an update before cleanup, but treat it as ephemeral and do not write state here.

When a plugin updates mid-session, hook commands, monitors, MCP servers, and LSP servers keep using the previous version's path. Run `/reload-plugins` to switch hooks, MCP servers, and LSP servers to the new path; monitors require a session restart.

`${CLAUDE_PLUGIN_DATA}` : a persistent directory for plugin state that survives updates. Use this for installed dependencies such as `node_modules` or Python virtual environments, generated code, caches, and any other files that should persist across plugin versions. The directory is created automatically the first time this variable is referenced.

`${CLAUDE_PROJECT_DIR}` : the project root. This is the same directory hooks receive in their `CLAUDE_PROJECT_DIR` variable. Use this to reference project-local scripts or config files. Wrap in

quotes to handle paths with spaces, for example `"${CLAUDE_PROJECT_DIR}/scripts/server.sh"`. MCP servers can also call the MCP `roots/list` request, which returns the directory Claude Code was launched from.

```
{
  "hooks": {
    "PostToolUse": [
      {
        "hooks": [
          {
            "type": "command",
            "command":
"\"${CLAUDE_PLUGIN_ROOT}\"/scripts/process.sh"
          }
        ]
      }
    ]
  }
}
```

Persistent data directory

The `${CLAUDE_PLUGIN_DATA}` directory resolves to `~/.claude/plugins/data/{id}/`, where `{id}` is the plugin identifier with characters outside `a-z`, `A-Z`, `0-9`, `_`, and `-` replaced by `-`. For a plugin installed as `formatter@my-marketplace`, the directory is `~/.claude/plugins/data/formatter-my-marketplace/`.

A common use is installing language dependencies once and reusing them across sessions and plugin updates. Because the data directory outlives any single plugin version, a check for directory existence alone cannot detect when an update changes the plugin's dependency manifest. The recommended pattern compares the bundled manifest against a copy in the data directory and reinstalls when they differ.

This `SessionStart` hook installs `node_modules` on the first run and again whenever a plugin update includes a changed `package.json`:

```
{
  "hooks": {
    "SessionStart": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "diff -q
\"${CLAUDE_PLUGIN_ROOT}/package.json\"
\"${CLAUDE_PLUGIN_DATA}/package.json\" >/dev/null 2>&1 || (cd
\"${CLAUDE_PLUGIN_DATA}\" && cp
\"${CLAUDE_PLUGIN_ROOT}/package.json\" . && npm install) || rm -f
\"${CLAUDE_PLUGIN_DATA}/package.json\""
          }
        ]
      }
    ]
  }
}
```

The `diff` exits nonzero when the stored copy is missing or differs from the bundled one, covering both first run and dependency-changing updates. If `npm install` fails, the trailing `rm` removes the copied manifest so the next session retries.

Scripts bundled in `${CLAUDE_PLUGIN_ROOT}` can then run against the persisted `node_modules`:

```
{
  "mcpServers": {
    "routines": {
      "command": "node",
      "args": ["${CLAUDE_PLUGIN_ROOT}/server.js"],
      "env": {
        "NODE_PATH": "${CLAUDE_PLUGIN_DATA}/node_modules"
      }
    }
  }
}
```

The data directory is deleted automatically when you uninstall the plugin from the last scope where it is installed. The `/plugin` interface shows the directory size and prompts before deleting. The CLI deletes by default; pass `--keep-data` to preserve it.

Plugin caching and file resolution

Plugins are specified in one of two ways:

- Through `claude --plugin-dir` or `claude --plugin-url`, for the duration of a session.
- Through a marketplace, installed for future sessions.

For security and verification purposes, Claude Code copies *marketplace* plugins to the user's local **plugin cache** (`~/.claude/plugins/cache`) rather than using them in-place. Understanding this behavior is important when developing plugins that reference external files.

Each installed version is a separate directory in the cache. When you update or uninstall a plugin, the previous version directory is marked as orphaned and removed automatically 7 days later. The grace period lets concurrent Claude Code sessions that already loaded the old version keep running without errors.

Claude's Glob and Grep tools skip orphaned version directories during searches, so file results don't include outdated plugin code.

Path traversal limitations

Installed plugins cannot reference files outside their directory. Paths that traverse outside the plugin root (such as `../shared-utils`) will not work after installation because those external files are not copied to the cache.

Share files within a marketplace with symlinks

If your plugin needs to share files with other parts of the same marketplace, you can create symbolic links inside your plugin directory. How a symlink is handled when the plugin is copied into the cache depends on where its target resolves:

- **Within the plugin's own directory:** the symlink is preserved as a relative symlink in the cache, so it keeps resolving to the copied target at runtime.
- **Elsewhere within the same marketplace:** the symlink is dereferenced. The target's content is copied into the cache in its place. This lets a meta-plugin's `skills/` directory link to skills defined by other plugins in the marketplace.
- **Outside the marketplace:** the symlink is skipped for security. This prevents plugins from pulling

arbitrary host files such as system paths into the cache.

For plugins installed with `--plugin-dir` or from a local path, only symlinks that resolve within the plugin's own directory are preserved. All others are skipped.

The following command creates a link from inside a marketplace plugin to a shared skill defined by a sibling plugin. On Windows, use `mklink /D` from an elevated Command Prompt or enable Developer Mode:

```
ln -s ../../shared-plugin/skills/foo ./skills/foo
```

This provides flexibility while maintaining the security benefits of the caching system.

Plugin directory structure

Standard plugin layout

A complete plugin follows this structure:

```
enterprise-plugin/
├── .claude-plugin/           # Metadata directory (optional)
│   └── plugin.json             # plugin manifest
├── skills/                   # Skills
│   ├── code-reviewer/
│   │   └── SKILL.md
│   └── pdf-processor/
│       ├── SKILL.md
│       └── scripts/
├── commands/                 # Skills as flat .md files
│   ├── status.md
│   └── logs.md
├── agents/                   # Subagent definitions
│   ├── security-reviewer.md
│   ├── performance-tester.md
│   └── compliance-checker.md
├── output-styles/            # Output style definitions
│   └── terse.md
├── themes/                   # Color theme definitions
│   └── dracula.json
├── monitors/                 # Background monitor
configurations
│   └── monitors.json
├── hooks/                    # Hook configurations
│   ├── hooks.json           # Main hook config
│   └── security-hooks.json  # Additional hooks
├── bin/                      # Plugin executables added to PATH
│   └── my-tool               # Invokable as bare command in
Bash tool
├── settings.json            # Default settings for the plugin
├── .mcp.json                # MCP server definitions
├── .lsp.json                # LSP server configurations
├── scripts/                 # Hook and utility scripts
│   ├── security-scan.sh
│   ├── format-code.py
│   └── deploy.js
├── LICENSE                  # License file
└── CHANGELOG.md             # Version history
```

⚠ The `.claude-plugin/` directory contains the `plugin.json` file. All other directories (commands/, agents/, skills/, output-styles/, themes/, monitors/, hooks/) must be at the plugin root, not inside `.claude-plugin/`.

A `CLAUDE.md` file at the plugin root is not loaded as project context. Plugins contribute context through skills, agents, and hooks rather than CLAUDE.md. To ship instructions that load into Claude's context, put them in a **skill**.

File locations reference

Component	Default Location	Purpose
Manifest	`.claude-plugin/plugin.json`	Plugin metadata and configuration (optional)
Skills	`skills/`	Skills with `<name>/SKILL.md` structure
Commands	`commands/`	Skills as flat Markdown files. Use `skills/` for new plugins
Agents	`agents/`	Subagent Markdown files
Output styles	`output-styles/`	Output style definitions
Themes	`themes/`	Color theme definitions
Hooks	`hooks/hooks.json`	Hook configuration
MCP servers	`.mcp.json`	MCP server definitions
LSP servers	`.lsp.json`	Language server configurations
Monitors	`monitors/monitors.json`	Background monitor configurations
Executables	`bin/`	Executables added to the Bash tool's `PATH`. Files here are invokable as bare commands in any Bash tool call while the plugin is enabled
Settings	`settings.json`	Default configuration applied when the plugin is enabled. Only the `agent` and `subagentStatusLine` keys are currently supported

CLI commands reference

Claude Code provides CLI commands for non-interactive plugin management, useful for scripting and automation.

plugin init

Scaffold a new plugin at `~/.claude/skills/<name>/`. On the next Claude Code session it loads automatically as `<name>@skills-dir` and appears in `/plugin` and `claude plugin list` with no install step.

See **Skills-directory plugins** for scope and trust requirements.

```
claude plugin init <name> [options]
```

Arguments:

- `<name>`: Plugin name. Becomes the skill namespace and the directory name under `~/.claude/skills/`, so it cannot contain spaces or path separators.

Options:

Option	Description	Default
`--description <text>`	Manifest description	
`--author <name>`	Author name	`git config user.name`
`--author-email <email>`	Author email	`git config user.email`
`--with <components...>`	Also scaffold component folders. Valid values: `skills`, `agents`, `hooks`, `mcp`, `lsp`, `output-style`, `channel`	
`-f, --force`	Overwrite an existing `.claude-plugin/` at the target	
`-h, --help`	Display help for command	

Aliases: `new`

Each `--with` value adds a starter file for that component, ready to edit:

Component	What it scaffolds
`skills`	An extra namespaced `<name>:example` skill alongside the default one
`agents`	An `agents/` subagent definition
`hooks`	A `hooks/hooks.json` with a sample event handler

Component	What it scaffolds
`mcp`	A `.mcp.json` with HTTP and stdio server examples
`lsp`	A `.lsp.json` language-server example
`output-style`	An `output-styles/<name>.md` that applies automatically while the plugin is enabled
`channel`	An MCP-based **channel**: a stdio server (`server.ts`), its `.mcp.json` , and a `package.json`

The scaffolded plugin uses the `@skills-dir` source rather than a marketplace. Admins can block this source with `strictKnownMarketplaces` or by adding `{"source": "skills-dir"}` to `blockedMarketplaces` in **managed settings**. When blocked, `plugin init` fails before writing.

Examples:

```
# Scaffold a minimal plugin
claude plugin init my-helper

# Scaffold with skill and hook folders
claude plugin init my-helper --with skills hooks

# Overwrite an existing scaffold
claude plugin init my-helper --force
```

plugin install

Install a plugin from available marketplaces.

```
claude plugin install <plugin> [options]
```

Arguments:

- `<plugin>` : Plugin name or `plugin-name@marketplace-name` for a specific marketplace

Options:

Option	Description	Default
`-s, --scope <scope>`	Installation scope: `user` , `project` , or `local`	`user`

Option	Description	Default
`-h, --help`	Display help for command	

Scope determines which settings file the installed plugin is added to. For example, `--scope project` writes to `enabledPlugins` in .claude/settings.json, making the plugin available to everyone who clones the project repository.

Examples:

```
# Install to user scope (default)
claude plugin install formatter@my-marketplace

# Install to project scope (shared with team)
claude plugin install formatter@my-marketplace --scope project

# Install to local scope (gitignored)
claude plugin install formatter@my-marketplace --scope local
```

plugin uninstall

Remove an installed plugin.

```
claude plugin uninstall <plugin> [options]
```

Arguments:

- `<plugin>`: Plugin name or `plugin-name@marketplace-name`

Options:

Option	Description	Default
`-s, --scope <scope>`	Uninstall from scope: `user`, `project`, or `local`	`user`
`--keep-data`	Preserve the plugin's **persistent data directory**	
`--prune`	Also remove auto-installed dependencies that no other plugin requires. See **plugin prune**	

Option	Description	Default
`-y, --yes`	Skip the `--prune` confirmation prompt. Required when stdin or stdout is not a TTY	
`-h, --help`	Display help for command	

Aliases: `remove`, `rm`

By default, uninstalling from the last remaining scope also deletes the plugin's `${CLAUDE_PLUGIN_DATA}` directory. Use `--keep-data` to preserve it, for example when reinstalling after testing a new version.

plugin prune

Remove auto-installed plugin dependencies that are no longer required by any installed plugin. Dependencies that Claude Code pulled in to satisfy another plugin's `dependencies` field are removed; plugins you installed directly are never touched.

```
claude plugin prune [options]
```

Options:

Option	Description	Default
`-s, --scope <scope>`	Prune at scope: `user`, `project`, or `local`	`user`
`--dry-run`	List what would be removed without removing anything	
`-y, --yes`	Skip the confirmation prompt. Required when stdin or stdout is not a TTY	
`-h, --help`	Display help for command	

Aliases: `autoremove`

The command lists orphaned dependencies and asks for confirmation before removing them. To remove a plugin and clean up its dependencies in one step, run `claude plugin uninstall <plugin> --prune`.

`claude plugin prune` requires Claude Code v2.1.121 or later.

plugin enable

Enable a disabled plugin. If the plugin declares **dependencies**, Claude Code enables them transitively at the same scope, and the command fails when a dependency is not installed.

```
claude plugin enable <plugin> [options]
```

Arguments:

- `<plugin>`: Plugin name or `plugin-name@marketplace-name`

Options:

Option	Description	Default
`-s, --scope <scope>`	Scope to enable: `user`, `project`, or `local`	`user`
`-h, --help`	Display help for command	

plugin disable

Disable a plugin without uninstalling it. Fails when another enabled plugin **depends on** the target. The error message includes a chained command that disables every dependent first.

```
claude plugin disable <plugin> [options]
```

Arguments:

- `<plugin>`: Plugin name or `plugin-name@marketplace-name`

Options:

Option	Description	Default
`-s, --scope <scope>`	Scope to disable: `user`, `project`, or `local`	`user`

Option	Description	Default
`-h, --help`	Display help for command	

plugin update

Update a plugin to the latest version.

```
claude plugin update <plugin> [options]
```

Arguments:

- `<plugin>` : Plugin name or `plugin-name@marketplace-name`

Options:

Option	Description	Default
`-s, --scope <scope>`	Scope to update: `user` , `project` , `local` , or `managed`	`user`
`-h, --help`	Display help for command	

plugin list

List installed plugins with their version, source marketplace, and enable status.

```
claude plugin list [options]
```

Options:

Option	Description	Default
`--json`	Output as JSON	
`--available`	Include available plugins from marketplaces. Requires `--json`	
`-h, --help`	Display help for command	

Within an interactive session, `/plugin list` prints the same listing inline. The interactive form accepts `--enabled` or `--disabled` to show only plugins in that state, and `ls` as a shorthand for `list`.

plugin details

Show a plugin's component inventory and projected token cost. The output lists all components the plugin contributes, grouped as Skills, Agents, Hooks, MCP servers, and LSP servers, along with an estimate of how many tokens it adds to each session. The Skills group includes both `skills/` and `commands/` entries.

```
claude plugin details <name>
```

Arguments:

- `<name>`: Plugin name or `plugin-name@marketplace-name`

Options:

Option	Description	Default
`-h, --help`	Display help for command	

The output shows two cost figures for each component:

- **Always-on:** tokens added to every session by the plugin's listing text, such as skill descriptions, agent descriptions, and command names, regardless of whether any component fires.
- **On-invoke:** tokens a component costs when it fires. Shown per component, not as a plugin total, because a typical session invokes only a subset of components.

This example shows what the output looks like for a plugin with two skills:

```
dependency-guard 1.2.0
  Dependency analysis for Claude Code sessions
  Source: dependency-guard@example-marketplace

Component inventory
  Skills (2)  scan-dependencies, review-changes
  Agents (0)
  Hooks (1)  (harness-only — no model context cost)
  MCP servers (0)
  LSP servers (0)

Projected token cost
  Always-on:   ~180 tok   added to every session

Per-component (rounded)
  component            always-on  on-invoke
  scan-dependencies        ~100      ~2400
  review-changes            ~80      ~1800

  On-invoke cost is paid each time a skill or agent fires.
  Token counts are estimates and may differ from actual usage.
```

The always-on total is computed via the `count_tokens` API for your active model. Per-component numbers are proportionally scaled from that total. If the API is unreachable, the command falls back to a character-based estimate.

plugin tag

Create a release git tag for the plugin in the current directory. Run from inside the plugin's folder. See **Tag plugin releases**.

```
claude plugin tag [options]
```

Options:

Option	Description	Default
`--push`	Push the tag to the remote after creating it	
`--dry-run`	Print what would be tagged without creating the tag	

Option	Description	Default
`-f, --force`	Create the tag even if the working tree is dirty or the tag already exists	
`-h, --help`	Display help for command	

Debugging and development tools

Debugging commands

Use `claude --debug` to see plugin loading details:

This shows:

- Which plugins are being loaded
- Any errors in plugin manifests
- Skill, agent, and hook registration
- MCP server initialization

Common issues

Issue	Cause	Solution
Plugin not loading	Invalid `plugin.json`	Run `claude plugin validate` or `/plugin validate` to check `plugin.json`, skill/agent/command frontmatter, and `hooks/hooks.json` for syntax and schema errors
Skills not appearing	Wrong directory structure	Ensure `skills/` or `commands/` is at the plugin root, not inside `.claude-plugin/`
Hooks not firing	Script not executable	Run `chmod +x script.sh`
MCP server fails	Missing `${CLAUDE_PLUGIN_ROOT}`	Use variable for all plugin paths
Path errors	Absolute paths used	All paths must be relative and start with `./`
LSP `Executable not found in $PATH`	Language server not installed	Install the binary (e.g., `npm install -g typescript-language-server typescript`)

Example error messages

Manifest validation errors:

- `Invalid JSON syntax: Unexpected token } in JSON at position 142`: check for missing commas, extra commas, or unquoted strings
- `Plugin has an invalid manifest file at .claude-plugin/plugin.json. Validation errors: name: Required`: a required field is missing
- `Plugin has a corrupt manifest file at .claude-plugin/plugin.json. JSON parse error: ...`: JSON syntax error

Plugin loading errors:

- `Warning: No commands found in plugin my-plugin custom directory: ./cmds. Expected .md files or SKILL.md in subdirectories.`: command path exists but contains no valid command files
- `Plugin directory not found at path: ./plugins/my-plugin. Check that the marketplace entry has the correct path.`: the `source` path in marketplace.json points to a non-existent directory
- `Plugin my-plugin has conflicting manifests: both plugin.json and marketplace entry specify components.`: remove duplicate component definitions or remove `strict: false` in marketplace entry

Hook troubleshooting

Hook script not executing:

1. Check the script is executable: `chmod +x ./scripts/your-script.sh`
2. Verify the shebang line: First line should be `#!/bin/bash` or `#!/usr/bin/env bash`
3. Check the path uses `${CLAUDE_PLUGIN_ROOT}`: `"command": "\"${CLAUDE_PLUGIN_ROOT}\"/scripts/your-script.sh"`
4. Test the script manually: `./scripts/your-script.sh`

Hook not triggering on expected events:

1. Verify the event name is correct (case-sensitive): `PostToolUse`, not `postToolUse`
2. Check the matcher pattern matches your tools: `"matcher": "Write|Edit"` for file operations
3. Confirm the hook type is valid: `command`, `http`, `mcp_tool`, `prompt`, or `agent`

MCP server troubleshooting

Server not starting:

1. Check the command exists and is executable
2. Verify all paths use `${CLAUDE_PLUGIN_ROOT}` variable
3. Check the MCP server logs: `claude --debug` shows initialization errors
4. Test the server manually outside of Claude Code

Server tools not appearing:

1. Ensure the server is properly configured in `.mcp.json` or `plugin.json`
2. Verify the server implements the MCP protocol correctly
3. Check for connection timeouts in debug output

Directory structure mistakes

Symptoms: Plugin loads but components (skills, agents, hooks) are missing.

Correct structure: Components must be at the plugin root, not inside `.claude-plugin/`. Only `plugin.json` belongs in `.claude-plugin/`.

```
my-plugin/
├── .claude-plugin/
│   └── plugin.json      ← Only manifest here
├── commands/            ← At root level
├── agents/              ← At root level
└── hooks/               ← At root level
```

If your components are inside `.claude-plugin/`, move them to the plugin root.

Debug checklist:

1. Run `claude --debug` and look for "loading plugin" messages
2. Check that each component directory is listed in the debug output
3. Verify file permissions allow reading the plugin files

Distribution and versioning reference

Version management

Claude Code uses the plugin's version as the cache key that determines whether an update is available. When you run `/plugin update` or auto-update fires, Claude Code computes the current version and skips the update if it matches what's already installed.

The version is resolved from the first of these that is set:

1. The `version` field in the plugin's `plugin.json`
2. The `version` field in the plugin's marketplace entry in `marketplace.json`
3. The git commit SHA of the plugin's source, for `github`, `url`, `git-subdir`, and relative-path sources in a git-hosted marketplace
4. `unknown`, for `npm` sources or local directories not inside a git repository

This gives you two ways to version a plugin:

Approach	How	Update behavior	Best for
Explicit version	Set `"version": "2.1.0"` in `plugin.json`	Users get updates only when you bump this field. Pushing new commits without bumping it has no effect, and `/plugin update` reports "already at the latest version".	Published plugins with stable release cycles
Commit-SHA version	Omit `version` from both `plugin.json` and the marketplace entry	Users get updates on every new commit to the plugin's git source	Internal or team plugins under active development

⚠ If you set `version` in `plugin.json`, you must bump it every time you want users to receive changes. Pushing new commits alone is not enough, because Claude Code sees the same version string and keeps the cached copy. If you're iterating quickly, leave `version` unset so the git commit SHA is used instead.

If you use explicit versions, follow **semantic versioning** (`MAJOR.MINOR.PATCH`): bump MAJOR for breaking changes, MINOR for new features, PATCH for bug fixes. Document changes in a `CHANGELOG.md`.

See also

- **Plugins** - Tutorials and practical usage
- **Plugin marketplaces** - Creating and managing marketplaces
- **Skills** - Skill development details
- **Subagents** - Agent configuration and capabilities
- **Hooks** - Event handling and automation
- **MCP** - External tool integration
- **Settings** - Configuration options for plugins